

Ch9: Linear Search vs Binary Search

Slide 1: Understanding Linear Search

When we search for a value in an array, the first question is:

How should we look for it?

The simplest method is **Linear Search**.

Linear search checks elements **one by one**, starting from the beginning of the array.

Think of searching for a student's name in a stack of papers that are **not arranged alphabetically**. You cannot jump directly to the answer because there is no useful order. You start with the first paper, check it, then the second, then the third, and continue until you find the name.

For example:

Array:

10 20 30 40 50 60 70

↑

Start here

If the target is 50, linear search works like this:

10 → 20 → 30 → 40 → 50

↑

Found

The program compares:

10 == 50 ? No

20 == 50 ? No

30 == 50 ? No

40 == 50 ? No

50 == 50 ? Yes

Important characteristics

- Checks one element at a time

- **Works with sorted or unsorted arrays**
- **Very simple to understand and implement**
- May need to check many elements before finding the target
- If the target is at the end, almost the entire array may be checked

For an array containing n elements, the worst case may require checking all n elements.

Key Idea

Linear Search does not need the data to be organized. It simply walks through the array from one end to the other.

Slide 2: Understanding Binary Search

Binary Search uses a completely different strategy.

Instead of checking elements one by one, it looks at the **middle element** and uses the ordering of the array to decide which half can be ignored.

Think of looking for a word in a dictionary.

You would not normally start at page 1 and turn every page. You open somewhere near the middle.

- Is the word before the middle? Search the left half.
- Is the word after the middle? Search the right half.
- Repeat.

Binary Search does exactly this with a **sorted array**.

Consider:

10 20 30 40 50 60 70
 ↑
 Middle

Suppose the target is 70.

First check:

Middle = 40

40 == 70 ? No

Because the array is sorted and $70 > 40$, everything to the **left of 40** can be ignored.

10 20 30 40 | 50 60 70
└────────── Search here

Now check the middle of the remaining section:

50 60 70
↑
Middle

$60 < 70$, so ignore the left part again:

50 60 | 70
└── Search here

Finally:

70
↑
Found

Why must the array be sorted?

Binary Search makes decisions such as:

Target < Middle

or

Target > Middle

These decisions are useful only because the elements are ordered.

For example:

10 20 30 40 50 60 70

If:

Target = 70
Middle = 40

we know:

$70 > 40$

Therefore, values before 40 cannot contain 70.

With an unsorted array:

50 10 70 20 40 60 30

seeing 40 in the middle tells us almost nothing about where 70 might be.

Key Idea

Binary Search is faster because it does not search everywhere. It repeatedly eliminates half of the remaining search area.

Slide 3: Linear Search vs Binary Search Example

Let's compare both methods using the same array.

Array:

10 20 30 40 50 60 70

Target = 70

Linear Search

Linear Search starts from the first element.

10 → 20 → 30 → 40 → 50 → 60 → 70

↑ ↑ ↑ ↑ ↑ ↑ ↑

Check each element

The comparisons are:

10 == 70 ? No

20 == 70 ? No

30 == 70 ? No

40 == 70 ? No

50 == 70 ? No

60 == 70 ? No

70 == 70 ? Yes

So the target is found after checking 7 elements.

Binary Search

Binary Search starts in the middle.

10 20 30 40 50 60 70

↑
Check

First:

$40 == 70$? No

$70 > 40$

Ignore:

10 20 30 40

Remaining:

50 60 70

Now check the middle:

50 60 70

↑
 $60 == 70$? No
 $70 > 60$

Ignore:

50 60

Remaining:

70

↑

Found.

So the search path is:

$40 \rightarrow 60 \rightarrow 70$

Only **3 elements** were checked.

Side-by-side view

Linear Search:

10 → 20 → 30 → 40 → 50 → 60 → 70
 ↑
 Found

Binary Search:

10 20 30 40 50 60 70
↓
40
↓
50 60 70
↓
60
↓
70

Search	First action	Next action
Linear	Check first element	Move to next element
Binary	Check middle element	Remove half of the search area

This is the fundamental difference.

Linear Search moves through the data. **Binary Search** reduces the data.

That distinction becomes increasingly important as the array becomes larger.

Slide 4: Comparing Linear Search and Binary Search

Both algorithms solve the same basic problem:

Find whether a target value exists in an array.

Feature	Linear Search	Binary Search
Data requirement	Sorted or unsorted	Must be sorted
Starting point	First element	Middle element
Search method	One by one	Divide the search area
Search reduction	Removes one element at a time	Removes about half each step
Implementation	Easier	Slightly more complex
Small arrays	Usually sufficient	Often unnecessary
Large sorted arrays	Can become slow	Much more efficient
Basic idea	Sequential checking	Repeated halving

Visual difference

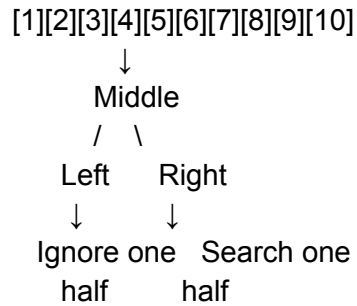
Suppose there are many elements.

Linear Search behaves approximately like:

[1][2][3][4][5][6][7][8][9][10]
↑
Check
↑
Check
↑
Check
↑
Check

It gradually moves forward.

Binary Search behaves more like:



Each decision can dramatically reduce the remaining work.

A useful way to remember it

Linear Search asks:

"Is it this element?"

Then:

"Is it the next element?"

Then:

"Is it the next one?"

And so on.

Binary Search asks:

"Which half could contain it?"

Then:

"Which half of that half?"

Then:

"Which half of that?"

This is why Binary Search becomes particularly powerful when the amount of data grows.

Key Idea

The biggest advantage of Binary Search comes from eliminating large portions of the array instead of checking every element.

Slide 5: Which Search Should You Use?

Choosing between Linear Search and Binary Search is not simply about asking:

"Which algorithm is faster?"

The better question is:

"What kind of data do I have?"

The structure and condition of the data determine which search method makes sense.

Use Linear Search when:

Data is small

↓

Linear Search

Linear Search is a good choice when:

- The array contains only a small number of elements.
- The data is **unsorted**.
- You want a simple implementation.
- Sorting the data first would not be worth the extra work.

Example:

45 12 89 23 7

This array is not sorted.

Linear Search can directly search it:

45 → 12 → 89 → 23 → 7

No preparation is required.

Use Binary Search when:

Data is sorted

+

Data is large

↓

Binary Search

Binary Search is a strong choice when:

- The array is already **sorted**.
- The amount of data is large.
- Searching needs to be efficient.
- The data can be kept in sorted order.

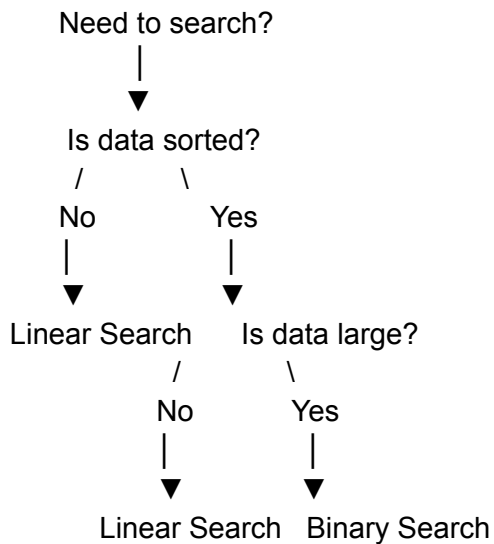
Example:

10 20 30 40 50 60 70 80 90

Binary Search can repeatedly use the middle element to eliminate half of the remaining possibilities.

Decision Framework

When you encounter a searching problem, think in this order:



The goal is not to memorize:

Linear = slow

Binary = fast

That is too simplistic.

Instead, remember the actual reasoning:

Linear Search



Can work on unsorted data



Simple and direct



Checks elements one by one

Binary Search



Requires sorted data



Uses the middle element



Eliminates half the search area



Efficient for large sorted data

Final Mental Model

Think of the two algorithms as two ways of looking for a person in a crowd.

Linear Search:

Person 1 → Person 2 → Person 3 → Person 4 → ...

You inspect people one at a time.

Binary Search:

Look at the middle



Choose the correct half



Look at the middle again



Choose the correct half



Repeat

Binary Search gets its advantage from **using information about the order of the data**.

Core Takeaway

Linear Search is simple and works even when data is unsorted. Binary Search is more efficient for large datasets, but it requires the data to be sorted.

The key comparison is therefore:

Linear Search → One by one

Binary Search → Half by half

And that single distinction explains most of the difference between them.